

Pong Reinforcement Learning with Self-Play Actor–Critic

High-Throughput Proximal Policy Optimization

Pranava Kumar

December 2025

Abstract

This project builds and deploys a reinforcement learning (RL) agent that plays Pong at a high level and is hosted as a web app. I started from a simple Double DQN baseline, discovered its limitations, and then redesigned the system around an actor–critic policy trained with PPO, reward shaping, vectorized environments, and self-play on an NVIDIA H200 GPU. The final agent reaches high rewards, long rallies, and a 100% win rate against a strong opponent, and its weights are exported to JSON and embedded into an HTML/JavaScript front end hosted on GitHub Pages.

Contents

1	Introduction	2
2	Project Scope and Work Overview	2
2.1	Project scope	2
2.2	Work done	3
3	Formulating Pong as a Reinforcement Learning Problem	3
3.1	MDP formulation	3
3.2	Reward shaping	4
4	Environment and Data Generation	4
4.1	Scalar Pong environment (DQN version)	4
4.2	Vectorized Pong environment for PPO	5
4.3	Self-play opponent	5
5	Algorithms and Model Architecture	5
5.1	Double DQN baseline	5
5.2	Actor–critic network	6
5.3	PPO objective and GAE	6
5.4	High-throughput implementation details	6
6	Evaluation	7
6.1	Training curves	7
6.2	Quantitative evaluation vs. simple AI	7
6.3	Qualitative behavior	7

7	Deployment as a Browser Game	7
7.1	Exporting PyTorch weights to JSON	7
7.2	HTML/JavaScript frontend	8
7.3	Hosting and integration	8
8	Challenges and Debugging Notes	9
9	Future Work	9
10	Conclusion	9
A	Double DQN Baseline Code	10
B	Actor–Critic PPO Training Code	12
C	HTML/JavaScript Frontend	29

1 Introduction

The goals of this project were:

1. Train a model that can play Pong head-to-head against a human.
2. Make the agent actually good at Pong, not just “moving roughly towards the ball.”
3. Host the trained model as a playable web app.
4. Deepen my own understanding of modern RL methods, self-play, and deployment pipelines.

Unlike a standard supervised learning problem with a fixed dataset, Pong as an RL task generates its own data by interacting with an environment. This makes the project a good playground for exploring exploration–exploitation tradeoffs, reward shaping, and performance engineering.

To keep everything accessible, I designed a lightweight scalar-state Pong environment, a Double DQN baseline, and then a much more sophisticated actor–critic agent that scales to tens of millions of steps in under two minutes on an NVIDIA H200.

The public demo is hosted at:

<https://pranavakcode.github.io/PongRL/>

All the code used for the project can viewed at:

<https://github.com/PranavaKCode/PongRL>

2 Project Scope and Work Overview

2.1 Project scope

At the very beginning I explicitly scoped the project around three deliverables:

1. A competent Pong agent that can play against me.
2. An agent that is *strong*, not just barely functional.

3. A browser-based Pong game where the backend policy is my trained model.

Throughout the project I also tracked secondary goals:

- Practice formulating RL problems precisely (state, action, reward, discount).
- Gain hands-on experience with self-play and reward shaping.
- Practice deployment: exporting PyTorch weights to JSON and re-implementing inference in JavaScript/TypeScript.

2.2 Work done

The concrete work I did can be grouped into four stages:

1. **DQN baseline.** Initially implemented a scalar-state Pong environment and trained a Double DQN agent with experience replay and ϵ -greedy exploration. Out of the box, without a large amount of fine-tuning the DQN family tends to perform better than the policy optimization family, AC including. So then what about with fine-tuning AC?
2. **Actor-critic redesign.** Replaced DQN with an actor-critic network trained with PPO, using a vectorized Pong environment and self-play.
3. **High-throughput training.** Optimized the environment and training loop to reach ≈ 1.2 million environment steps per second on an NVIDIA H200.
4. **Deployment.** Exported the final actor weights to JSON, created an HTML/JavaScript frontend that re-implements the network and environment, and hosted everything via GitHub Pages.

In the rest of the report I first describe how Pong is formulated as an RL problem, then dive into the algorithms, engineering decisions, and evaluation, and finally discuss future directions such as a polished React/TypeScript interface.

3 Formulating Pong as a Reinforcement Learning Problem

3.1 MDP formulation

I treat Pong as a Markov decision process (MDP) with:

- **State s_t :** For the final actor-critic system each environment observation is a 6-dimensional vector

$$s_t = [x_{\text{ball}}, y_{\text{ball}}, v_x, v_y, y_{\text{opponent}}, y_{\text{self}}],$$

normalized to approximately $[-1, 1]$.¹

- **Actions a_t :** Discrete actions for the right paddle: 0 = stay, 1 = up, 2 = down.
- **Transition dynamics:** Deterministic motion of paddles and ball plus paddle and wall collisions, implemented in a vectorized environment (see Section 4).

¹Normalization is important so that the scale of inputs is consistent across Python and JavaScript implementations.

- **Reward r_t :** A shaped reward that combines (1) distance-to-ball improvements, (2) bonuses for successful returns, (3) rally length bonuses, and (4) scoring and win bonuses.
- **Discount factor γ :** $\gamma = 0.99$.

Episodes terminate when either player reaches 11 points.

3.2 Reward shaping

Raw Pong rewards (only scoring events) are sparse and noisy. To make learning more stable and faster I used several reward components, computed per environment per time step:

1. **Distance reward.** Let d_t be the vertical distance between the agent paddle center and the ball. Then:

$$r_t^{\text{dist}} = \alpha \cdot (d_{t-1} - d_t),$$

with $\alpha = 2.0$. Moving closer to the ball yields positive reward, moving away yields negative reward.

2. **Hit and rally reward.** When the agent successfully returns the ball:

$$r_t^{\text{hit}} = r_{\text{hit}} + \beta \cdot \text{rally_length},$$

with $r_{\text{hit}} = 1.0$ and $\beta = 0.002$.

3. **Score and win reward.** When the agent scores or loses a point:

$$r_t^{\text{score}} = \begin{cases} +5.0 & \text{if agent scores,} \\ -5.0 & \text{if opponent scores,} \end{cases}$$

and at the end of a game a win bonus of +10.0 or loss penalty of -10.0 is applied.

The final reward is the sum of these components. This shaping makes the gradient signal much denser and pushes the policy toward tracking the ball, maintaining long rallies, and actually winning games.

4 Environment and Data Generation

4.1 Scalar Pong environment (DQN version)

For the initial Double DQN experiment I built a very simple scalar environment (**PongEnv**, Listing A) that operates in continuous coordinates on $[0, 1] \times [0, 1]$:

- Ball position (x, y) and velocity $(\dot{x}, \dot{y})$.
- A single controlled paddle on the left, with height 0.2.
- Reflecting top and bottom walls.

The environment computes distance-based reward, paddle hit bonuses, and negative rewards when the AI misses the ball. It returns a 5-dimensional state vector:

$$[y_{\text{ai}}, x_{\text{ball}}, y_{\text{ball}}, \dot{x}, \dot{y}].$$

Episodes are collected by running this environment until the ball passes the left wall.

4.2 Vectorized Pong environment for PPO

For the actor-critic system I redesigned the environment as a fully vectorized class, `PongEnv`, that simulates N independent games in parallel on the GPU:

- **Number of environments:** $N = 8192$.
- **Logical resolution:** 84×84 units, used purely for geometry and rendering.
- **State representation:** 6 scalar features as described earlier, normalized before being passed into the network.
- **Vectorized physics:** Ball and paddles are represented as batched tensors, and all collisions are computed using elementwise tensor operations.

The vectorized design allows collection of $N \times T$ transitions per rollout, with $T = 128$ steps, giving a batch size of

$$B = N \cdot T = 8192 \cdot 128 = 1,048,576$$

steps per PPO update.

4.3 Self-play opponent

The environment supports two modes for the left paddle:

- A simple tracking AI that moves toward the ball.
- A frozen copy of the current policy (self-play).

During training I maintain a small pool of past policies and periodically update the opponent to be either the latest policy or a randomly selected older snapshot. This avoids overfitting to a single fixed opponent and creates a lightweight league training effect.

5 Algorithms and Model Architecture

5.1 Double DQN baseline

The baseline is a standard Double DQN setup:

- A feedforward network with two hidden layers of size 32 and ReLU activations, mapping the 5D state to 3 Q-values.
- Experience replay buffer of length 5000.
- ϵ -greedy exploration with exponential decay ($\epsilon_{\text{start}} = 1.0$, $\epsilon_{\text{min}} = 0.05$, decay factor 0.997).
- Double DQN target: the same network is used for action selection and evaluation in a simplified DDQN update.

Although this model learned to at least move toward the ball and occasionally return it, qualitatively it was slow to react and easy to exploit. The screenshots of the web demo show the AI lagging behind fast horizontal motion and missing many balls.

This motivated the switch to a more expressive actor-critic approach.

5.2 Actor–critic network

The final network architecture is an actor–critic model implemented in `ActorCritic` (Listing B). It consists of:

- **Shared trunk:** Three fully connected layers of dimension 256 with LayerNorm and ReLU:

$$h = \text{ReLU}(\text{LN}(W_3 \text{ReLU}(\text{LN}(W_2 \text{ReLU}(\text{LN}(W_1 x)))))).$$

- **Actor head:** A linear layer mapping h to 3 action logits.
- **Critic head:** A linear layer mapping h to a scalar value estimate $V(s_t)$.

Weights are initialized orthogonally and the actor head uses a smaller gain to keep early policies near-uniform.

5.3 PPO objective and GAE

Training uses Proximal Policy Optimization (PPO) with generalized advantage estimation (GAE):

- For each rollout we compute advantages:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t),$$

$$A_t = \delta_t + \gamma \lambda (1 - d_t) A_{t+1},$$

where d_t is the done flag, $\gamma = 0.99$, and $\lambda = 0.95$.

- We compute returns $R_t = A_t + V(s_t)$.
- For each minibatch the PPO objective is

$$L_{\text{PPO}} = \mathbb{E} [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)],$$

where $r_t(\theta) = \exp(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t))$ and $\epsilon = 0.2$.

The total loss combines policy, value, and entropy terms:

$$L = -L_{\text{PPO}} + c_v L_{\text{value}} - c_e H[\pi_\theta],$$

with $c_v = 0.5$ and $c_e = 0.01$.

5.4 High-throughput implementation details

Key engineering details that made the training fast and stable:

- **Vectorized environment** on the GPU to avoid Python loops.
- **Mixed precision** training using `torch.cuda.amp` when available.
- **Large batch size** ($\approx 10^6$ transitions per update) with 4 PPO epochs and 32 minibatches.
- **H200 GPU** accessible via Apptainer and CUDA, which required some manual setup:
 - Explicitly inserting the user-level `site-packages` directory into `sys.path` so `pip` installs are visible.
 - Avoiding `torch.compile` because it was not working correctly in this environment.

A quick benchmark shows an estimated throughput above one million environment steps per second and a total training time of about 1.4 minutes for 100 million steps.

6 Evaluation

6.1 Training curves

During training I log several metrics:

- Episode reward (smoothed over recent episodes).
- Win rate against past versions of the policy (self-play).
- Average rally length.
- Average distance to the ball.
- Policy entropy.

The training plots show the episode reward rising sharply and then slowly drifting upward, rally lengths reaching around 80–100, average distance to the ball stabilizing at low values, and entropy gradually decreasing as the policy becomes more confident.

6.2 Quantitative evaluation vs. simple AI

After training I evaluate the final policy against a simple tracking AI for 100 games using the `evaluate` function in Listing B. The result is:

100 wins, 0 losses (100.0% win rate).

This indicates that the policy is significantly stronger than a straightforward hand-coded opponent.

6.3 Qualitative behavior

Qualitatively, the agent:

- Tracks the ball smoothly and centers the paddle relative to the expected impact point.
- Is able to survive very long rallies, especially when the ball speed is moderate.
- Occasionally fails on extreme edge cases (for example, when I set the initial ball speed very high), which is expected given the training regime.

I also tuned the initial ball speed used in the web demo, turning it into an explicit hyperparameter (`INITIAL_BALL_SPEED`) so that the game is fun for humans and not instantly unwinnable.

7 Deployment as a Browser Game

7.1 Exporting PyTorch weights to JSON

To make the model portable, I export a subset of the weights into a JSON file:

- Only the shared trunk and actor head are needed for action selection in the browser.

- Each layer’s weights and biases are detached, moved to CPU, and converted to nested Python lists.
- The resulting dictionary is saved to `pong_brain.json`.

The final JSON file is only on the order of tens of kilobytes, so it is practical to ship directly to the browser.

7.2 HTML/JavaScript frontend

The web frontend (Listing C) performs the following steps:

1. Fetch `pong_brain.json` using `fetch()` and store its contents in a global `weights` object.
2. Re-implement the neural network in pure JavaScript:
 - A generic `dense` function for matrix–vector multiply plus bias.
 - A simple `relu` function.
 - A basic layer normalization function mirroring the PyTorch architecture.
 - An `getAction` function that applies the three shared layers and the actor head and takes the argmax over logits.
3. Maintain a lightweight Pong environment in JavaScript that matches the training dynamics:
 - A logical 84×84 coordinate system for ball and paddles.
 - Physics with wall bounces, paddle collisions, and speed clamping.
 - A keyboard-controlled left paddle and neural-network-controlled right paddle.
4. Render frames to an HTML5 canvas at 600×400 pixels, including score HUD and dashed center line.

The inputs to `getAction` are normalized in the same way as in the PyTorch environment, which is critical to avoid distribution shift.

7.3 Hosting and integration

The HTML and JSON files are committed to the GitHub repository and served through GitHub Pages. Because everything is static (no backend server), the deployment is effectively just pushing to the `main` branch of:

<https://github.com/PranavaKCode/PongRL>

with the site available at:

<https://pranavakcode.github.io/PongRL/>

8 Challenges and Debugging Notes

Some of the main challenges and how I worked around them:

- **Hardware and environment setup.** Accessing the NVIDIA H200 required working through Apptainer and CUDA; several Python packages were missing from the container.
 - Solution: explicitly prepend `~.local/lib/python3.12/site-packages` to `sys.path`, which made user-level `pip` installs visible.
- **torch.compile instability.** `torch.compile` did not behave reliably in this environment.
 - Solution: disable `torch.compile` entirely and rely on good vectorization and mixed precision instead.
- **Debugging a billion-step training run.** With 100 million to 1 billion environment steps, mistakes are expensive.
 - Solution: use an LLM to help generate targeted unit tests, intermediate print statements, and comments, and run small debugging configurations before full-scale runs.
- **Reward shaping correctness.** It is easy to accidentally create degenerate incentives (for example, maximizing distance reward while losing points).
 - Solution: inspect all reward components, monitor rally length and distance to ball, and manually play against intermediate policies.

9 Future Work

I am close to finishing a more polished app version built with HTML, JSON, TypeScript, and React on top of the same exported weights. Planned improvements include:

- A clean React UI with menus, difficulty sliders, and live metrics (for example, rally length and estimated value).
- Multiple difficulty levels by mixing the RL agent with simpler policies or by temperature-scaling the logits.
- Optional pixel-based observations to explore convolutional architectures, while keeping the current scalar-based system as a fast baseline.
- Saving replays for analysis and possibly training secondary models (for example, a supervised model that predicts human actions).

10 Conclusion

This project started as a small DQN implementation and grew into a full fledged RL system that can take advantage of big compute: a vectorized Pong environment, a PPO-style actor-critic agent with self-play and reward shaping, high-throughput training on an NVIDIA H200, and a browser-based deployment via JSON-exported weights and an HTML/JavaScript frontend.

Beyond getting an good agent, the project gave me experience in RL formulation, performance engineering, and end-to-end deployment from PyTorch to the web.

A Double DQN Baseline Code

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import random
6 from collections import deque
7 import json
8
9 class PongEnv:
10     def __init__(self):
11         self.height = 1.0
12         self.width = 1.0
13         self.paddle_h = 0.2
14         self.reset()
15
16     def reset(self):
17         self.ball_x = 0.5
18         self.ball_y = 0.5
19         self.ball_dx = random.choice([-0.03, 0.03])
20         self.ball_dy = random.uniform(-0.02, 0.02)
21         self.ai_y = 0.4
22         self.opponent_y = 0.4
23         self.score = 0
24         self.done = False
25         return self.get_state()
26
27     def get_state(self):
28         # State: [AI_Y, Ball_X, Ball_Y, Ball_dX, Ball_dY]
29         return np.array(
30             [self.ai_y, self.ball_x, self.ball_y, self.ball_dx, self.ball_dy],
31             dtype=np.float32
32         )
33
34     def step(self, action):
35         # Action: 0=Stay, 1=Up, 2=Down
36         prev_dist = abs((self.ai_y + self.paddle_h/2) - self.ball_y)
37
38         speed = 0.04
39         if action == 1:
40             self.ai_y -= speed
41         if action == 2:
42             self.ai_y += speed
43         self.ai_y = np.clip(self.ai_y, 0, 1 - self.paddle_h)
44
45         # Move Ball
46         self.ball_x += self.ball_dx
47         self.ball_y += self.ball_dy
48
49         # Bounce Top/Bottom
50         if self.ball_y <= 0 or self.ball_y >= 1:
51             self.ball_dy *= -1
52
53         # REWARD SHAPING
54         new_dist = abs((self.ai_y + self.paddle_h/2) - self.ball_y)
55         reward = (prev_dist - new_dist) * 10
56
```

```

57         # Hit AI Paddle (Left side)
58         if (
59             self.ball_x <= 0.02 and
60             (self.ai_y - 0.05 <= self.ball_y <= self.ai_y + self.paddle_h + 0.05)
61         ):
62             self.ball_dx *= -1
63             self.ball_dx *= 1.05 # Speed up game
64             self.ball_x = 0.03
65             reward = 5.0 # Big reward for return
66
67         # Hit Wall (AI Missed)
68         if self.ball_x < 0:
69             reward = -10
70             self.done = True
71
72         # Hit Opponent Wall (AI Scored/Opponent Missed)
73         if self.ball_x > 1:
74             self.ball_dx *= -1
75             self.ball_x = 0.99
76
77         return self.get_state(), reward, self.done
78
79 class DQN(nn.Module):
80     def __init__(self):
81         super(DQN, self).__init__()
82         self.fc1 = nn.Linear(5, 32)
83         self.fc2 = nn.Linear(32, 32)
84         self.out = nn.Linear(32, 3)
85
86     def forward(self, x):
87         x = torch.relu(self.fc1(x))
88         x = torch.relu(self.fc2(x))
89         return self.out(x)
90
91 env = PongEnv()
92 model = DQN()
93 optimizer = optim.Adam(model.parameters(), lr=0.005)
94 criterion = nn.MSELoss()
95
96 episodes = 1500
97 gamma = 0.95
98 epsilon = 1.0
99 epsilon_decay = 0.997
100 replay_buffer = deque(maxlen=5000)
101 batch_size = 64
102
103 print("Training Upgraded AI (approx 2 mins)...")
104
105 for ep in range(episodes):
106     state = env.reset()
107     done = False
108     total_reward = 0
109
110     while not done:
111         if random.random() < epsilon:
112             action = random.randint(0, 2)
113         else:
114             with torch.no_grad():
115                 q = model(torch.tensor(state).unsqueeze(0))

```

```

116         action = torch.argmax(q).item()
117
118     next_state, reward, done = env.step(action)
119     replay_buffer.append((state, action, reward, next_state, done))
120     state = next_state
121     total_reward += reward
122
123     if len(replay_buffer) > batch_size:
124         batch = random.sample(replay_buffer, batch_size)
125         s, a, r, ns, d = zip(*batch)
126
127         s_t = torch.tensor(s, dtype=torch.float32)
128         a_t = torch.tensor(a, dtype=torch.int64).unsqueeze(1)
129         r_t = torch.tensor(r, dtype=torch.float32).unsqueeze(1)
130         ns_t = torch.tensor(ns, dtype=torch.float32)
131         d_t = torch.tensor(d, dtype=torch.float32).unsqueeze(1)
132
133         # Double DQN logic (simplified)
134         target_q = r_t + gamma * torch.max(model(ns_t), 1)[0].unsqueeze(1) *
            (1 - d_t)
135         current_q = model(s_t).gather(1, a_t)
136
137         loss = criterion(current_q, target_q)
138         optimizer.zero_grad()
139         loss.backward()
140         optimizer.step()
141
142     epsilon = max(0.05, epsilon * epsilon_decay)
143
144     if ep % 200 == 0:
145         print(f"Episode {ep}: Reward {total_reward:.2f} (Epsilon {epsilon:.2f})")
146
147 print("Training Complete!")
148
149 model_weights = {
150     "w1": model.fc1.weight.detach().tolist(),
151     "b1": model.fc1.bias.detach().tolist(),
152     "w2": model.fc2.weight.detach().tolist(),
153     "b2": model.fc2.bias.detach().tolist(),
154     "w3": model.out.weight.detach().tolist(),
155     "b3": model.out.bias.detach().tolist()
156 }
157 weights_json = json.dumps(model_weights)
158 print("Brain exported for browser!")

```

B Actor–Critic PPO Training Code

```

1  """
2  Pong RL v2.7
3  """
4
5  import sys, os
6  sys.path.insert(0, os.path.expanduser('~/.local/lib/python3.12/site-packages'))
7
8  import torch
9  import torch.nn as nn

```

```

10 from torch.distributions import Categorical
11 import time
12 from dataclasses import dataclass
13 from typing import Optional, List
14 from collections import deque
15 import matplotlib.pyplot as plt
16 from IPython.display import clear_output
17 import warnings
18 warnings.filterwarnings('ignore')
19
20 try:
21     from torch.cuda.amp import autocast, GradScaler
22     AMP_AVAILABLE = True
23 except ImportError:
24     AMP_AVAILABLE = False
25
26 print(f"PyTorch: {torch.__version__}")
27 print(f"CUDA: {torch.cuda.is_available()}")
28 print(f"AMP: {AMP_AVAILABLE}")
29
30
31 # =====
32 # CONFIGURATION
33 # =====
34
35 @dataclass
36 class Config:
37     # Environment
38     num_envs: int = 8192
39     width: int = 84
40     height: int = 84
41     paddle_height: int = 12
42     paddle_width: int = 4
43     ball_size: int = 3
44     paddle_speed: float = 1.5
45     ball_speed: float = 1.5
46     max_ball_speed: float = 4.0
47
48     # PPO Hyperparameters
49     learning_rate: float = 3e-4
50     gamma: float = 0.99
51     gae_lambda: float = 0.95
52     clip_epsilon: float = 0.2
53     value_coef: float = 0.5
54     entropy_coef: float = 0.01
55     max_grad_norm: float = 0.5
56
57     # Training
58     total_timesteps: int = 100_000_000
59     rollout_steps: int = 128
60     num_minibatches: int = 32
61     update_epochs: int = 4
62
63     # Self-play
64     opponent_update_freq: int = 10
65     latest_opponent_prob: float = 0.5
66     max_opponent_pool: int = 10
67
68     # Reward shaping

```

```

69     distance_reward_scale: float = 2.0
70     hit_reward: float = 1.0
71     rally_reward_scale: float = 0.002
72     score_reward: float = 5.0
73     win_bonus: float = 10.0
74
75     # Optimization
76     use_mixed_precision: bool = True
77
78     # Logging
79     plot_interval: int = 10
80     save_interval: int = 50
81
82     @property
83     def batch_size(self) -> int:
84         return self.num_envs * self.rollout_steps
85
86     @property
87     def minibatch_size(self) -> int:
88         return self.batch_size // self.num_minibatches
89
90
91     # =====
92     # NEURAL NETWORK
93     # =====
94
95     class ActorCritic(nn.Module):
96         def __init__(self, obs_dim: int = 6, act_dim: int = 3, hidden: int = 256):
97             super().__init__()
98
99             self.shared = nn.Sequential(
100                 nn.Linear(obs_dim, hidden),
101                 nn.LayerNorm(hidden),
102                 nn.ReLU(),
103                 nn.Linear(hidden, hidden),
104                 nn.LayerNorm(hidden),
105                 nn.ReLU(),
106                 nn.Linear(hidden, hidden),
107                 nn.LayerNorm(hidden),
108                 nn.ReLU(),
109             )
110
111             self.actor = nn.Linear(hidden, act_dim)
112             self.critic = nn.Linear(hidden, 1)
113
114             # Initialize weights
115             for module in self.modules():
116                 if isinstance(module, nn.Linear):
117                     nn.init.orthogonal_(module.weight, gain=1.0)
118                     nn.init.zeros_(module.bias)
119             nn.init.orthogonal_(self.actor.weight, gain=0.01)
120             nn.init.orthogonal_(self.critic.weight, gain=1.0)
121
122         def forward(self, x):
123             h = self.shared(x)
124             return self.actor(h), self.critic(h).squeeze(-1)
125
126         def get_action_and_value(self, x, action=None):
127             logits, value = self(x)

```

```

128         dist = Categorical(logits=logits)
129         if action is None:
130             action = dist.sample()
131         return action, dist.log_prob(action), dist.entropy(), value
132
133
134 # =====
135 # PONG ENVIRONMENT WITH DISTANCE REWARD
136 # =====
137
138 class PongEnv:
139     """
140     Vectorized Pong environment with distance-based reward shaping.
141     """
142
143     def __init__(self, cfg: Config, device: torch.device):
144         self.cfg = cfg
145         self.device = device
146         self.n = cfg.num_envs
147
148         # Dimensions
149         self.W = float(cfg.width)
150         self.H = float(cfg.height)
151         self.ph = float(cfg.paddle_height)
152         self.pw = float(cfg.paddle_width)
153         self.bs = float(cfg.ball_size)
154
155         # Ball state
156         self.ball_x = torch.zeros(self.n, device=device)
157         self.ball_y = torch.zeros(self.n, device=device)
158         self.ball_vx = torch.zeros(self.n, device=device)
159         self.ball_vy = torch.zeros(self.n, device=device)
160
161         # Paddle positions (y-coordinate of center)
162         self.left_y = torch.zeros(self.n, device=device)
163         self.right_y = torch.zeros(self.n, device=device)
164
165         # For distance reward
166         self.prev_dist = torch.zeros(self.n, device=device)
167
168         # Game state
169         self.rally = torch.zeros(self.n, device=device, dtype=torch.int32)
170         self.score_left = torch.zeros(self.n, device=device, dtype=torch.int32)
171         self.score_right = torch.zeros(self.n, device=device, dtype=torch.int32)
172         self.ep_reward = torch.zeros(self.n, device=device)
173         self.ep_length = torch.zeros(self.n, device=device, dtype=torch.int32)
174
175         # Opponent model
176         self.opponent = None
177
178         # Initialize
179         self.reset()
180
181     def set_opponent(self, model: nn.Module):
182         """Set the opponent model for self-play."""
183         self.opponent = model
184
185     def _reset_ball(self, idx: torch.Tensor, direction: Optional[torch.Tensor] =
None):

```

```

186     """Reset ball position and velocity for given environment indices."""
187     m = idx.shape[0]
188     if m == 0:
189         return
190
191     # Center position
192     self.ball_x[idx] = self.W / 2
193     self.ball_y[idx] = self.H / 2
194
195     # Random angle (-45 to +45 degrees)
196     angle = (torch.rand(m, device=self.device) - 0.5) * 1.5
197
198     # Random or specified direction
199     if direction is None:
200         direction = torch.sign(torch.rand(m, device=self.device) - 0.5)
201         direction = torch.where(direction == 0, torch.ones_like(direction),
202                                direction)
203
204     speed = self.cfg.ball_speed
205     self.ball_vx[idx] = direction * speed * torch.cos(angle).abs().clamp(min=0.5)
206     self.ball_vy[idx] = speed * torch.sin(angle) * 0.5
207
208     self.rally[idx] = 0
209
210 def reset(self, idx: Optional[torch.Tensor] = None) -> torch.Tensor:
211     """Reset environments. If idx is None, reset all."""
212     if idx is None:
213         idx = torch.arange(self.n, device=self.device)
214
215     self._reset_ball(idx)
216
217     # Center paddles
218     self.left_y[idx] = self.H / 2
219     self.right_y[idx] = self.H / 2
220
221     # Reset distance tracking
222     self.prev_dist[idx] = torch.abs(self.right_y[idx] - self.ball_y[idx])
223
224     # Reset scores
225     self.score_left[idx] = 0
226     self.score_right[idx] = 0
227     self.ep_reward[idx] = 0
228     self.ep_length[idx] = 0
229
230     return self._get_obs()
231
232 def _get_obs(self, flip: bool = False) -> torch.Tensor:
233     """
234     Get observation tensor.
235
236     State: [ball_x, ball_y, ball_vx, ball_vy, opponent_y, self_y]
237     All normalized to roughly [-1, 1]
238
239     If flip=True, return from left paddle's perspective (for opponent).
240     """
241     if flip:
242         # Opponent's view (left paddle) - flip x coordinates
243         return torch.stack([

```

```

243         (self.W - self.ball_x) / self.W * 2 - 1, # Flipped ball x
244         self.ball_y / self.H * 2 - 1,
245         -self.ball_vx / self.cfg.max_ball_speed, # Flipped velocity
246         self.ball_vy / self.cfg.max_ball_speed,
247         self.right_y / self.H * 2 - 1,           # Agent is opponent's
            opponent
248         self.left_y / self.H * 2 - 1,           # Self
249     ], dim=1)
250 else:
251     # Agent's view (right paddle)
252     return torch.stack([
253         self.ball_x / self.W * 2 - 1,
254         self.ball_y / self.H * 2 - 1,
255         self.ball_vx / self.cfg.max_ball_speed,
256         self.ball_vy / self.cfg.max_ball_speed,
257         self.left_y / self.H * 2 - 1,           # Opponent
258         self.right_y / self.H * 2 - 1,         # Self
259     ], dim=1)
260
261 @torch.no_grad()
262 def step(self, action: torch.Tensor):
263     """
264     Execute one step for all environments.
265
266     Action: 0 = stay, 1 = up, 2 = down
267
268     Returns: obs, reward, done, info
269     """
270     # Store previous distance for reward shaping
271     self.prev_dist = torch.abs(self.right_y - self.ball_y)
272
273     ps = self.cfg.paddle_speed
274
275     # ===== OPPONENT (LEFT PADDLE) =====
276     if self.opponent is not None:
277         opp_obs = self._get_obs(flip=True)
278         opp_logits, _ = self.opponent(opp_obs)
279         opp_action = opp_logits.argmax(dim=1)
280     else:
281         # Simple tracking AI as fallback
282         diff = self.ball_y - self.left_y
283         opp_action = torch.zeros(self.n, device=self.device, dtype=torch.long)
284         opp_action = torch.where(diff < -2, torch.ones_like(opp_action),
            opp_action)
285         opp_action = torch.where(
286             diff > 2, torch.full_like(opp_action, 2), opp_action
287         )
288
289     # Move opponent paddle
290     opp_move = torch.zeros(self.n, device=self.device)
291     opp_move = torch.where(opp_action == 1, -ps, opp_move)
292     opp_move = torch.where(opp_action == 2, ps, opp_move)
293     self.left_y = (self.left_y + opp_move).clamp(self.ph / 2, self.H - self.ph
        / 2)
294
295     # ===== AGENT (RIGHT PADDLE) =====
296     agent_move = torch.zeros(self.n, device=self.device)
297     agent_move = torch.where(action == 1, -ps, agent_move)
298     agent_move = torch.where(action == 2, ps, agent_move)

```

```

299     self.right_y = (self.right_y + agent_move).clamp(
300         self.ph / 2, self.H - self.ph / 2
301     )
302
303     # ===== BALL PHYSICS =====
304     self.ball_x = self.ball_x + self.ball_vx
305     self.ball_y = self.ball_y + self.ball_vy
306
307     # Top/bottom wall collision
308     hit_top = self.ball_y < self.bs
309     hit_bottom = self.ball_y > self.H - self.bs
310     hit_wall = hit_top | hit_bottom
311     self.ball_vy = torch.where(hit_wall, -self.ball_vy, self.ball_vy)
312     self.ball_y = self.ball_y.clamp(self.bs, self.H - self.bs)
313
314     # ===== PADDLE COLLISIONS =====
315     # Right paddle (agent)
316     right_paddle_x = self.W - self.pw
317     hit_right = (
318         (self.ball_x >= right_paddle_x - self.bs) &
319         (self.ball_vx > 0) &
320         (torch.abs(self.ball_y - self.right_y) <= self.ph / 2 + self.bs)
321     )
322
323     # Left paddle (opponent)
324     hit_left = (
325         (self.ball_x <= self.pw + self.bs) &
326         (self.ball_vx < 0) &
327         (torch.abs(self.ball_y - self.left_y) <= self.ph / 2 + self.bs)
328     )
329
330     # Bounce off paddles
331     hit_paddle = hit_right | hit_left
332     self.ball_vx = torch.where(hit_paddle, -self.ball_vx * 1.02, self.ball_vx)
333
334     # Add spin based on where ball hits paddle
335     paddle_y = torch.where(hit_right, self.right_y, self.left_y)
336     relative_hit = torch.where(
337         hit_paddle,
338         (self.ball_y - paddle_y) / (self.ph / 2),
339         torch.zeros_like(self.ball_y)
340     )
341     self.ball_vy = self.ball_vy + relative_hit * 0.4
342
343     # Keep ball in bounds after paddle hit (prevent tunneling)
344     self.ball_x = torch.where(
345         hit_right, right_paddle_x - self.bs - 0.5, self.ball_x
346     )
347     self.ball_x = torch.where(
348         hit_left, self.pw + self.bs + 0.5, self.ball_x
349     )
350
351     # Clamp ball speed
352     speed = torch.sqrt(self.ball_vx ** 2 + self.ball_vy ** 2)
353     too_fast = speed > self.cfg.max_ball_speed
354     scale = self.cfg.max_ball_speed / (speed + 1e-8)
355     self.ball_vx = torch.where(too_fast, self.ball_vx * scale, self.ball_vx)
356     self.ball_vy = torch.where(too_fast, self.ball_vy * scale, self.ball_vy)
357

```

```

358     # Update rally counter
359     self.rally = self.rally + 1
360
361     # ===== REWARD CALCULATION =====
362     new_dist = torch.abs(self.right_y - self.ball_y)
363     dist_reward = (self.prev_dist - new_dist) * self.cfg.distance_reward_scale
364     reward = dist_reward
365
366     # Hit bonus + rally bonus
367     rally_bonus = self.rally.float() * self.cfg.rally_reward_scale
368     reward = torch.where(
369         hit_right, reward + self.cfg.hit_reward + rally_bonus, reward
370     )
371
372     # Scoring
373     scored_left = self.ball_x < 0
374     scored_right = self.ball_x > self.W
375
376     reward = torch.where(scored_right, reward + self.cfg.score_reward, reward)
377     reward = torch.where(scored_left, reward - self.cfg.score_reward, reward)
378
379     # Update scores
380     self.score_left = self.score_left + scored_left.int()
381     self.score_right = self.score_right + scored_right.int()
382
383     # Game over at 11 points
384     done = (self.score_left >= 11) | (self.score_right >= 11)
385
386     # Win/loss bonus
387     reward = torch.where(
388         done & (self.score_right > self.score_left),
389         reward + self.cfg.win_bonus,
390         reward
391     )
392     reward = torch.where(
393         done & (self.score_left > self.score_right),
394         reward - self.cfg.win_bonus,
395         reward
396     )
397
398     # Track episode stats
399     self.ep_reward = self.ep_reward + reward
400     self.ep_length = self.ep_length + 1
401
402     # ===== COLLECT INFO =====
403     info = {
404         'ep_rewards': self.ep_reward[done].tolist() if done.any() else [],
405         'ep_lengths': self.ep_length[done].tolist() if done.any() else [],
406         'wins': int((self.score_right[done] > self.score_left[done]).sum()) if
407             done.any() else 0,
408         'losses': int((self.score_left[done] > self.score_right[done]).sum())
409             if done.any() else 0,
410         'avg_rally': float(self.rally.float().mean()),
411         'avg_dist': float(new_dist.mean()),
412     }
413
414     # Reset ball after scoring
415     scored = scored_left | scored_right
416     if scored.any():

```

```

415         scored_idx = torch.where(scored)[0]
416         direction = torch.where(
417             scored_right[scored_idx],
418             -torch.ones(scored_idx.shape[0], device=self.device),
419             torch.ones(scored_idx.shape[0], device=self.device)
420         )
421         self._reset_ball(scored_idx, direction)
422
423     # Full reset for finished games
424     if done.any():
425         self.reset(torch.where(done)[0])
426
427     return self._get_obs(), reward, done, info
428
429 def render(self, idx: int = 0):
430     """Render a single environment to numpy array."""
431     h, w = int(self.H), int(self.W)
432     img = torch.zeros((h, w, 3), dtype=torch.uint8, device='cpu')
433
434     # Center line
435     for y in range(0, h, 4):
436         img[y:min(y+2, h), w//2-1:w//2+1] = 50
437
438     # Ball (white)
439     bx = int(self.ball_x[idx].item())
440     by = int(self.ball_y[idx].item())
441     b = int(self.bs)
442     y1, y2 = max(0, by - b), min(h, by + b)
443     x1, x2 = max(0, bx - b), min(w, bx + b)
444     img[y1:y2, x1:x2] = torch.tensor([255, 255, 255], dtype=torch.uint8)
445
446     # Left paddle (blue)
447     ly = int(self.left_y[idx].item())
448     p = int(self.ph // 2)
449     pw = int(self.pw)
450     y1, y2 = max(0, ly - p), min(h, ly + p)
451     img[y1:y2, 0:pw] = torch.tensor([100, 150, 255], dtype=torch.uint8)
452
453     # Right paddle (green)
454     ry = int(self.right_y[idx].item())
455     y1, y2 = max(0, ry - p), min(h, ry + p)
456     img[y1:y2, w - pw:w] = torch.tensor([100, 255, 150], dtype=torch.uint8)
457
458     return img.numpy()
459
460
461 # =====
462 # PPO TRAINER
463 # =====
464
465 class PPOTrainer:
466     def __init__(self, cfg: Config, device: torch.device):
467         self.cfg = cfg
468         self.device = device
469
470         print("Initializing trainer...")
471
472     # Environment
473     self.env = PongEnv(cfg, device)

```

```

474
475 # Agent model
476 self.model = ActorCritic().to(device)
477 print(f" Model parameters: {sum(p.numel() for p in self.model.parameters
    ()):,}")
478
479 # Opponent model (copy of agent)
480 self.opponent = ActorCritic().to(device)
481 self.opponent.load_state_dict(self.model.state_dict())
482 self.opponent.eval()
483 self.env.set_opponent(self.opponent)
484
485 # Optimizer
486 self.optimizer = torch.optim.Adam(self.model.parameters(), lr=cfg.
    learning_rate)
487
488 # Mixed precision
489 self.use_amp = AMP_AVAILABLE and cfg.use_mixed_precision
490 self.scaler = GradScaler() if self.use_amp else None
491 print(f" Mixed precision: {self.use_amp}")
492
493 # Rollout buffers (pre-allocated)
494 T, N = cfg.rollout_steps, cfg.num_envs
495 self.buf_obs = torch.zeros((T, N, 6), device=device)
496 self.buf_act = torch.zeros((T, N), device=device, dtype=torch.long)
497 self.buf_rew = torch.zeros((T, N), device=device)
498 self.buf_done = torch.zeros((T, N), device=device)
499 self.buf_val = torch.zeros((T, N), device=device)
500 self.buf_logp = torch.zeros((T, N), device=device)
501
502 # Training stats
503 self.global_step = 0
504 self.num_updates = 0
505 self.ep_rewards = deque(maxlen=500)
506 self.win_history = deque(maxlen=50)
507
508 # Logging
509 self.log = {
510     'step': [],
511     'reward': [],
512     'winrate': [],
513     'rally': [],
514     'dist': [],
515     'entropy': [],
516     'fps': [],
517 }
518
519 # Opponent pool for league training
520 self.opponent_pool: List[dict] = []
521
522 print(" Initialization complete!")
523
524 def update_opponent(self):
525     """Update opponent from current or historical policy."""
526     state_dict = {k: v.clone() for k, v in self.model.state_dict().items()}
527
528     if len(self.opponent_pool) < self.cfg.max_opponent_pool:
529         self.opponent_pool.append(state_dict)
530     else:

```

```

531         idx = self.num_updates % self.cfg.max_opponent_pool
532         self.opponent_pool[idx] = state_dict
533
534         if torch.rand(1).item() < self.cfg.latest_opponent_prob or len(self.
535             opponent_pool) < 3:
536             self.opponent.load_state_dict(self.model.state_dict())
537         else:
538             idx = torch.randint(0, len(self.opponent_pool), (1,)).item()
539             self.opponent.load_state_dict(self.opponent_pool[idx])
540
541         print(f"                Opponent updated (pool size: {len(self.opponent_pool)
542             })")
543
544     @torch.no_grad()
545     def collect_rollout(self):
546         """Collect experience by running policy in environment."""
547         self.model.eval()
548         obs = self.env._get_obs()
549
550         wins, losses = 0, 0
551         rallies = []
552         distances = []
553
554         for t in range(self.cfg.rollout_steps):
555             self.buf_obs[t] = obs
556
557             if self.use_amp:
558                 with autocast():
559                     action, log_prob, _, value = self.model.get_action_and_value(
560                         obs)
561             else:
562                 action, log_prob, _, value = self.model.get_action_and_value(obs)
563
564             self.buf_act[t] = action
565             self.buf_logp[t] = log_prob
566             self.buf_val[t] = value
567
568             # Step environment
569             obs, reward, done, info = self.env.step(action)
570
571             self.buf_rew[t] = reward
572             self.buf_done[t] = done.float()
573
574             self.ep_rewards.extend(info['ep_rewards'])
575             wins += info['wins']
576             losses += info['losses']
577             rallies.append(info['avg_rally'])
578             distances.append(info['avg_dist'])
579
580             self.global_step += self.cfg.num_envs
581
582         if self.use_amp:
583             with autocast():
584                 _, _, _, next_value = self.model.get_action_and_value(obs)
585         else:
586             _, _, _, next_value = self.model.get_action_and_value(obs)
587
588         advantages = torch.zeros_like(self.buf_rew)
589         last_gae = 0

```

```

587
588     for t in reversed(range(self.cfg.rollout_steps)):
589         if t == self.cfg.rollout_steps - 1:
590             next_val = next_value
591         else:
592             next_val = self.buf_val[t + 1]
593
594         non_terminal = 1.0 - self.buf_done[t]
595         delta = self.buf_rew[t] + self.cfg.gamma * next_val * non_terminal -
            self.buf_val[t]
596         last_gae = delta + self.cfg.gamma * self.cfg.gae_lambda * non_terminal
            * last_gae
597         advantages[t] = last_gae
598
599     returns = advantages + self.buf_val
600
601     if wins + losses > 0:
602         self.win_history.append(wins / (wins + losses))
603
604     return {
605         'advantages': advantages,
606         'returns': returns,
607         'avg_rally': sum(rallies) / len(rallies),
608         'avg_dist': sum(distances) / len(distances),
609         'wins': wins,
610         'losses': losses,
611     }
612
613 def update(self, advantages: torch.Tensor, returns: torch.Tensor):
614     """Perform PPO update."""
615     self.model.train()
616
617     B = self.cfg.batch_size
618     b_obs = self.buf_obs.reshape(B, 6)
619     b_act = self.buf_act.reshape(B)
620     b_logp = self.buf_logp.reshape(B)
621     b_adv = advantages.reshape(B)
622     b_ret = returns.reshape(B)
623
624     b_adv = (b_adv - b_adv.mean()) / (b_adv.std() + 1e-8)
625
626     total_entropy = 0
627     count = 0
628
629     for epoch in range(self.cfg.update_epochs):
630         perm = torch.randperm(B, device=self.device)
631
632         for start in range(0, B, self.cfg.minibatch_size):
633             idx = perm[start:start + self.cfg.minibatch_size]
634
635             if self.use_amp:
636                 with autocast():
637                     _, new_logp, entropy, new_val = self.model.
                        get_action_and_value(
638                         b_obs[idx], b_act[idx]
639                     )
640
641                     ratio = (new_logp - b_logp[idx]).exp()
642                     pg_loss1 = -b_adv[idx] * ratio

```

```

643         pg_loss2 = -b_adv[idx] * ratio.clamp(
644             1 - self.cfg.clip_epsilon,
645             1 + self.cfg.clip_epsilon
646         )
647         pg_loss = torch.max(pg_loss1, pg_loss2).mean()
648
649         val_loss = 0.5 * ((new_val - b_ret[idx]) ** 2).mean()
650         ent_loss = entropy.mean()
651         loss = pg_loss + self.cfg.value_coef * val_loss - self.cfg
            .entropy_coef * ent_loss
652
653         self.optimizer.zero_grad()
654         self.scaler.scale(loss).backward()
655         self.scaler.unscale_(self.optimizer)
656         nn.utils.clip_grad_norm_(self.model.parameters(), self.cfg.
            max_grad_norm)
657         self.scaler.step(self.optimizer)
658         self.scaler.update()
659     else:
660         _, new_logp, entropy, new_val = self.model.
            get_action_and_value(
661                 b_obs[idx], b_act[idx]
662             )
663
664         ratio = (new_logp - b_logp[idx]).exp()
665         pg_loss1 = -b_adv[idx] * ratio
666         pg_loss2 = -b_adv[idx] * ratio.clamp(
667             1 - self.cfg.clip_epsilon,
668             1 + self.cfg.clip_epsilon
669         )
670         pg_loss = torch.max(pg_loss1, pg_loss2).mean()
671
672         val_loss = 0.5 * ((new_val - b_ret[idx]) ** 2).mean()
673         ent_loss = entropy.mean()
674         loss = pg_loss + self.cfg.value_coef * val_loss - self.cfg.
            entropy_coef * ent_loss
675
676         self.optimizer.zero_grad()
677         loss.backward()
678         nn.utils.clip_grad_norm_(self.model.parameters(), self.cfg.
            max_grad_norm)
679         self.optimizer.step()
680
681         total_entropy += ent_loss.item()
682         count += 1
683
684     self.num_updates += 1
685     return total_entropy / count
686
687 def train(self):
688     """Main training loop."""
689     total_updates = self.cfg.total_timesteps // self.cfg.batch_size
690
691     print(f"\n{'='*65}")
692     print(f"      Pong Self-Play Training (with Distance Reward)")
693     print(f"{'='*65}")
694     print(f"  Device:           {self.device}")
695     print(f"  Environments:     {self.cfg.num_envs:,}")
696     print(f"  Batch size:      {self.cfg.batch_size:,}")

```

```

697     print(f"    Total updates:    {total_updates}")
698     print(f"    Total steps:        {self.cfg.total_timesteps:,}")
699     print(f"    Opponent update: every {self.cfg.opponent_update_freq} updates")
700     print(f"    Distance reward: {self.cfg.distance_reward_scale}")
701     print(f"    Rally bonus:        {self.cfg.rally_reward_scale}")
702     print(f"{'='*65}\n")
703
704     start_time = time.time()
705
706     for update in range(1, total_updates + 1):
707         update_start = time.time()
708
709         rollout = self.collect_rollout()
710         entropy = self.update(rollout['advantages'], rollout['returns'])
711
712         if update % self.cfg.opponent_update_freq == 0:
713             self.update_opponent()
714
715         update_time = time.time() - update_start
716         fps = self.cfg.batch_size / update_time
717
718         avg_reward = sum(self.ep_rewards) / max(1, len(self.ep_rewards))
719         winrate = sum(self.win_history) / max(1, len(self.win_history)) * 100
720
721         self.log['step'].append(self.global_step)
722         self.log['reward'].append(avg_reward)
723         self.log['winrate'].append(winrate)
724         self.log['rally'].append(rollout['avg_rally'])
725         self.log['dist'].append(rollout['avg_dist'])
726         self.log['entropy'].append(entropy)
727         self.log['fps'].append(fps)
728
729         elapsed = time.time() - start_time
730         eta = elapsed / update * (total_updates - update)
731
732         print(
733             f"[{update:3d}/{total_updates}] "
734             f"Step {self.global_step:>10,} | "
735             f"FPS {fps:>9,.0f} | "
736             f"Rew {avg_reward:>7.2f} | "
737             f"Win {winrate:>5.1f}% | "
738             f"Rally {rollout['avg_rally']:>5.1f} | "
739             f"Dist {rollout['avg_dist']:>4.1f} | "
740             f"Ent {entropy:.3f} | "
741             f"ETA {eta/60:.1f}m"
742         )
743
744         if update % self.cfg.plot_interval == 0:
745             clear_output(wait=True)
746             self._plot()
747             print(f"\n    Update {update}/{total_updates} | Step {self.
748                 global_step:,}")
749
750         if update % self.cfg.save_interval == 0:
751             self._save(f"pong_checkpoint_{self.global_step}.pt")
752
753     total_time = time.time() - start_time
754     print(f"\n{'='*65}")
755     print(f"    Training Complete!")

```

```

755     print(f"    Total time:  {total_time/60:.1f} minutes")
756     print(f"    Avg FPS:      {sum(self.log['fps'])/len(self.log['fps']):.0f}")
757     print(f"    Final rally: {self.log['rally'][-1]:.1f}")
758     print(f"{'='*65}")
759
760     self._save("pong_final.pt")
761
762     def _plot(self):
763         """Plot training progress."""
764         if len(self.log['reward']) < 2:
765             return
766
767         fig, axes = plt.subplots(2, 3, figsize=(15, 8))
768
769         axes[0, 0].plot(self.log['step'], self.log['reward'], 'b-', alpha=0.7)
770         axes[0, 0].set_title('Episode Reward')
771         axes[0, 0].set_xlabel('Steps')
772         axes[0, 0].grid(True, alpha=0.3)
773
774         axes[0, 1].plot(self.log['step'], self.log['winrate'], 'g-', alpha=0.7)
775         axes[0, 1].axhline(50, color='r', linestyle='--', alpha=0.5, label='50%')
776         axes[0, 1].set_title('Win Rate vs Past Self')
777         axes[0, 1].set_xlabel('Steps')
778         axes[0, 1].set_ylim(0, 100)
779         axes[0, 1].legend()
780         axes[0, 1].grid(True, alpha=0.3)
781
782         axes[0, 2].plot(self.log['step'], self.log['rally'], 'm-', alpha=0.7)
783         axes[0, 2].set_title('Avg Rally Length')
784         axes[0, 2].set_xlabel('Steps')
785         axes[0, 2].grid(True, alpha=0.3)
786
787         axes[1, 0].plot(self.log['step'], self.log['dist'], 'c-', alpha=0.7)
788         axes[1, 0].set_title('Avg Distance to Ball')
789         axes[1, 0].set_xlabel('Steps')
790         axes[1, 0].grid(True, alpha=0.3)
791
792         axes[1, 1].plot(self.log['step'], self.log['entropy'], 'orange', alpha=
793                        =0.7)
794         axes[1, 1].set_title('Policy Entropy')
795         axes[1, 1].set_xlabel('Steps')
796         axes[1, 1].grid(True, alpha=0.3)
797
798         try:
799             img = self.env.render(0)
800             axes[1, 2].imshow(img)
801             sl = int(self.env.score_left[0].item())
802             sr = int(self.env.score_right[0].item())
803             axes[1, 2].set_title(f'Game: Opponent {sl} - {sr} Agent')
804         except Exception as e:
805             axes[1, 2].text(0.5, 0.5, f'Render error:\n{e}', ha='center', va='
806                        center')
807         axes[1, 2].axis('off')
808
809         plt.tight_layout()
810         plt.show()
811
812     def _save(self, path: str):
813         """Save model checkpoint."""

```

```

812         torch.save({
813             'model': self.model.state_dict(),
814             'optimizer': self.optimizer.state_dict(),
815             'step': self.global_step,
816             'updates': self.num_updates,
817             'config': self.cfg,
818         }, path)
819         print(f"             Saved: {path}")
820
821     def load(self, path: str):
822         """Load model checkpoint."""
823         ckpt = torch.load(path, map_location=self.device)
824         self.model.load_state_dict(ckpt['model'])
825         if 'optimizer' in ckpt:
826             self.optimizer.load_state_dict(ckpt['optimizer'])
827         self.global_step = ckpt.get('step', 0)
828         self.num_updates = ckpt.get('updates', 0)
829         print(f"             Loaded: {path}")
830
831
832 # =====
833 # EVALUATION
834 # =====
835
836 def evaluate(trainer: PPOTrainer, num_games: int = 100):
837     """Evaluate agent against simple tracking AI."""
838     print(f"\n Evaluating against simple AI ({num_games} games)...")
839
840     trainer.model.eval()
841
842     old_opponent = trainer.env.opponent
843     trainer.env.opponent = None
844
845     trainer.env.reset()
846     wins, losses = 0, 0
847
848     with torch.no_grad():
849         while wins + losses < num_games:
850             obs = trainer.env._get_obs()
851             action, _, _, _ = trainer.model.get_action_and_value(obs)
852             _, _, _, info = trainer.env.step(action)
853             wins += info['wins']
854             losses += info['losses']
855
856     trainer.env.opponent = old_opponent
857
858     winrate = wins / (wins + losses) * 100
859     print(f"     Results: {wins}W - {losses}L ({winrate:.1f}% win rate)")
860
861     return winrate
862
863
864 # =====
865 # EXPORT FOR BROWSER
866 # =====
867
868 def export_for_browser(trainer, filename="pong_brain.json"):
869     """Export model weights for JavaScript."""
870     model = trainer.model

```

```

871
872 weights = {
873     "shared_0_w": model.shared[0].weight.detach().cpu().tolist(),
874     "shared_0_b": model.shared[0].bias.detach().cpu().tolist(),
875     "shared_3_w": model.shared[3].weight.detach().cpu().tolist(),
876     "shared_3_b": model.shared[3].bias.detach().cpu().tolist(),
877     "shared_6_w": model.shared[6].weight.detach().cpu().tolist(),
878     "shared_6_b": model.shared[6].bias.detach().cpu().tolist(),
879     "actor_w": model.actor.weight.detach().cpu().tolist(),
880     "actor_b": model.actor.bias.detach().cpu().tolist(),
881 }
882
883 with open(filename, 'w') as f:
884     import json
885     json.dump(weights, f)
886
887 import json
888 print(f" Exported to {filename}")
889 print(f"    Size: {len(json.dumps(weights)) / 1024:.1f} KB")
890
891 # =====
892 # MAIN
893 # =====
894
895
896 def main():
897     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
898     print(f"\t\t\t\t\t Device: {device}")
899
900     if device.type == 'cuda':
901         props = torch.cuda.get_device_properties(0)
902         print(f"        GPU: {props.name}")
903         print(f"        Memory: {props.total_memory / 1e9:.1f} GB")
904
905     cfg = Config(
906         num_envs=8192,
907         total_timesteps=100_000_000,
908         rollout_steps=128,
909         opponent_update_freq=10,
910         latest_opponent_prob=0.5,
911         distance_reward_scale=2.0,
912         hit_reward=1.0,
913         rally_reward_scale=0.002,
914         score_reward=5.0,
915         win_bonus=10.0,
916         plot_interval=10,
917         save_interval=50,
918     )
919
920     print(f"\t\t\t\t\t Quick benchmark...")
921     temp_env = PongEnv(Config(num_envs=4096), device)
922     temp_model = ActorCritic().to(device)
923
924     obs = temp_env.reset()
925     start = time.time()
926     for _ in range(50):
927         with torch.no_grad():
928             action, _, _, _ = temp_model.get_action_and_value(obs)
929             obs, _, _, _ = temp_env.step(action)

```

```

930     if device.type == 'cuda':
931         torch.cuda.synchronize()
932         elapsed = time.time() - start
933         fps = 50 * 4096 / elapsed
934         print(f"    Estimated FPS: {fps:,.0f}")
935         print(f"    Est. time for 100M steps: {100_000_000 / fps / 60:.1f} minutes")
936
937     del temp_env, temp_model
938     if device.type == 'cuda':
939         torch.cuda.empty_cache()
940
941     trainer = PPOTrainer(cfg, device)
942     trainer.train()
943     evaluate(trainer, num_games=100)
944     export_for_browser(trainer)
945
946     return trainer
947
948 if __name__ == "__main__":
949     trainer = main()
950

```

C HTML/JavaScript Frontend

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="utf-8">
5  <title>Pong RL Demo</title>
6  <style>
7      body {
8          background-color: #111;
9          color: #eee;
10         font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",
            sans-serif;
11         margin: 0;
12         padding: 20px;
13         display: flex;
14         flex-direction: column;
15         align-items: center;
16     }
17     .hud {
18         width: 600px;
19         display: flex;
20         justify-content: space-between;
21         font-size: 24px;
22         font-weight: 700;
23         margin-bottom: 10px;
24     }
25     .hud-left { color: #4facfe; } /* AI */
26     .hud-right { color: #ff5f6d; } /* YOU */
27     canvas {
28         background: #000;
29         border: 2px solid #555;
30         box-shadow: 0 0 40px rgba(0,0,0,0.8);
31         cursor: none;

```

```

32     }
33     .instructions {
34         margin-top: 10px;
35         color: #888;
36         font-size: 14px;
37     }
38 </style>
39 </head>
40 <body>
41
42 <div class="hud">
43     <span class="hud-left">AI: <span id="sAi">0</span></span>
44     <span class="hud-right">YOU: <span id="sP1">0</span></span>
45 </div>
46
47 <canvas id="gameCanvas" width="600" height="400"></canvas>
48
49 <div class="instructions">
50     <div>
51         Controls: <b>Arrow Keys</b>
52         <span style="background:#333; padding:2px 8px; border-radius:4px; color:#
53             fff; font-size:0.8em;">    </span>
54         <span style="background:#333; padding:2px 8px; border-radius:4px; color:#
55             fff; font-size:0.8em;">    </span>
56     </div>
57     Status: <b style="color:#4facfe;">Neural Network Active</b>
58 </div>
59
60 <script>
61 // ----- CONFIG -----
62 const INITIAL_BALL_SPEED = 1.0;
63
64 // ----- LOAD WEIGHTS -----
65 let weights = null;
66
67 fetch("pong_brain.json")
68     .then(r => r.json())
69     .then(w => {
70         weights = w;
71         startGame();
72     })
73     .catch(err => {
74         console.error("Failed to load pong_brain.json", err);
75         alert("Error loading pong_brain.json    check console.");
76     });
77
78 function startGame() {
79     function dense(input, w, b) {
80         const out = new Array(b.length).fill(0);
81         for (let i = 0; i < b.length; i++) {
82             let s = 0;
83             for (let j = 0; j < input.length; j++) s += input[j] * w[i][j];
84             out[i] = s + b[i];
85         }
86         return out;
87     }
88

```

```

89 function relu(arr) { return arr.map(x => Math.max(0, x)); }
90
91 function layerNorm(arr) {
92     const mean = arr.reduce((a,b)=>a+b,0) / arr.length;
93     const varc = arr.reduce((a,b)=>a+(b-mean)*(b-mean),0) / arr.length;
94     const std = Math.sqrt(varc + 1e-5);
95     return arr.map(x => (x - mean) / std);
96 }
97
98 function getAction(obs) {
99     let x = dense(obs, weights.shared_0_w, weights.shared_0_b);
100     x = relu(layerNorm(x));
101
102     x = dense(x, weights.shared_3_w, weights.shared_3_b);
103     x = relu(layerNorm(x));
104
105     x = dense(x, weights.shared_6_w, weights.shared_6_b);
106     x = relu(layerNorm(x));
107
108     const logits = dense(x, weights.actor_w, weights.actor_b);
109
110     let best = 0, bestVal = -Infinity;
111     for (let i = 0; i < logits.length; i++) {
112         if (logits[i] > bestVal) {
113             bestVal = logits[i];
114             best = i;
115         }
116     }
117     return best; // 0=stay, 1=up, 2=down
118 }
119
120 const LOGICAL_W = 84;
121 const LOGICAL_H = 84;
122 const PADDLE_H = 12;
123 const PADDLE_W = 4;
124 const BALL_SIZE = 3;
125 const MAX_BALL_SPEED = 4.0;
126 const PADDLE_SPEED = 1.5;
127
128 const canvas = document.getElementById("gameCanvas");
129 const ctx = canvas.getContext("2d");
130 const CANVAS_W = canvas.width;
131 const CANVAS_H = canvas.height;
132 const SX = CANVAS_W / LOGICAL_W;
133 const SY = CANVAS_H / LOGICAL_H;
134
135 const sAiEl = document.getElementById("sAi");
136 const sPlEl = document.getElementById("sPl");
137
138 let state = {
139     ballX: LOGICAL_W / 2,
140     ballY: LOGICAL_H / 2,
141     ballVX: -INITIAL_BALL_SPEED,
142     ballVY: 0.5,
143     leftY: LOGICAL_H / 2,
144     rightY: LOGICAL_H / 2,
145     scoreL: 0,
146     scoreR: 0
147 };

```

```

148
149 let keys = {up:false, down:false};
150 window.addEventListener("keydown", e => {
151     if (e.key === "ArrowUp") { keys.up = true; e.preventDefault(); }
152     if (e.key === "ArrowDown") { keys.down = true; e.preventDefault(); }
153 });
154 window.addEventListener("keyup", e => {
155     if (e.key === "ArrowUp") keys.up = false;
156     if (e.key === "ArrowDown") keys.down = false;
157 });
158
159 function resetBall(winner) {
160     state.ballX = LOGICAL_W / 2;
161     state.ballY = LOGICAL_H / 2;
162     const dir = winner === "left" ? 1 : -1;
163     state.ballVX = dir * INITIAL_BALL_SPEED;
164     state.ballVY = (Math.random() - 0.5) * 2;
165 }
166
167 function update() {
168     if (keys.up) state.leftY -= PADDLE_SPEED;
169     if (keys.down) state.leftY += PADDLE_SPEED;
170     state.leftY = Math.max(PADDLE_H/2, Math.min(LOGICAL_H - PADDLE_H/2, state.
        leftY));
171
172     const obs = [
173         (state.ballX / LOGICAL_W) * 2 - 1,
174         (state.ballY / LOGICAL_H) * 2 - 1,
175         state.ballVX / MAX_BALL_SPEED,
176         state.ballVY / MAX_BALL_SPEED,
177         (state.leftY / LOGICAL_H) * 2 - 1,
178         (state.rightY / LOGICAL_H) * 2 - 1
179     ];
180     const action = getAction(obs);
181     if (action === 1) state.rightY -= PADDLE_SPEED;
182     if (action === 2) state.rightY += PADDLE_SPEED;
183     state.rightY = Math.max(PADDLE_H/2, Math.min(LOGICAL_H - PADDLE_H/2, state.
        rightY));
184
185     state.ballX += state.ballVX;
186     state.ballY += state.ballVY;
187
188     if (state.ballY <= BALL_SIZE || state.ballY >= LOGICAL_H - BALL_SIZE) {
189         state.ballVY *= -1;
190         state.ballY = Math.max(BALL_SIZE, Math.min(LOGICAL_H - BALL_SIZE, state.
            ballY));
191     }
192
193     if (state.ballX <= PADDLE_W + BALL_SIZE && state.ballVX < 0) {
194         if (Math.abs(state.ballY - state.leftY) <= PADDLE_H/2 + BALL_SIZE) {
195             state.ballVX *= -1.05;
196             const rel = (state.leftY - state.ballY) / (PADDLE_H/2);
197             state.ballVY -= rel * 0.5;
198         }
199     }
200
201     if (state.ballX >= LOGICAL_W - PADDLE_W - BALL_SIZE && state.ballVX > 0) {
202         if (Math.abs(state.ballY - state.rightY) <= PADDLE_H/2 + BALL_SIZE) {
203             state.ballVX *= -1.05;

```

```

204         const rel = (state.rightY - state.ballY) / (PADDLE_H/2);
205         state.ballVY -= rel * 0.5;
206     }
207 }
208
209 const speed = Math.hypot(state.ballVX, state.ballVY);
210 if (speed > MAX_BALL_SPEED) {
211     const r = MAX_BALL_SPEED / speed;
212     state.ballVX *= r;
213     state.ballVY *= r;
214 }
215
216 if (state.ballX < 0) {
217     state.scoreR++;
218     sAiEl.textContent = state.scoreR;
219     resetBall("right");
220 }
221 if (state.ballX > LOGICAL_W) {
222     state.scoreL++;
223     sPlEl.textContent = state.scoreL;
224     resetBall("left");
225 }
226 }
227
228 function draw() {
229     ctx.fillStyle = "#111";
230     ctx.fillRect(0, 0, CANVAS_W, CANVAS_H);
231
232     ctx.strokeStyle = "#333";
233     ctx.setLineDash([10, 15]);
234     ctx.beginPath();
235     ctx.moveTo(CANVAS_W/2, 0);
236     ctx.lineTo(CANVAS_W/2, CANVAS_H);
237     ctx.stroke();
238     ctx.setLineDash([]);
239
240     ctx.fillStyle = "#4facfe";
241     ctx.fillRect(
242         (LOGICAL_W - PADDLE_W) * SX,
243         (state.rightY - PADDLE_H/2) * SY,
244         PADDLE_W * SX,
245         PADDLE_H * SY
246     );
247
248     ctx.fillStyle = "#ff5f6d";
249     ctx.fillRect(
250         0,
251         (state.leftY - PADDLE_H/2) * SY,
252         PADDLE_W * SX,
253         PADDLE_H * SY
254     );
255
256     ctx.fillStyle = "#fff";
257     ctx.beginPath();
258     ctx.arc(state.ballX * SX, state.ballY * SY, BALL_SIZE * ((SX+SY)/2), 0, Math
        .PI*2);
259     ctx.fill();
260 }
261

```

```
262     function loop() {
263         update();
264         draw();
265         requestAnimationFrame(loop);
266     }
267
268     loop();
269 }
270 </script>
271 </body>
272 </html>
```